Tokenisation & Word Embeddings

Jour 4 — Matin

Julien Rolland

Formation M2 Développement Fullstack

Jour 4

Plan du module

1. Du texte aux nombres
2. One-Hot vs embeddings denses
3. Word2Vec & propriétés
4. Embeddings en pratique
5. Visualisation & limites

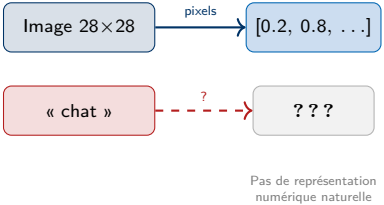
Le défi du langage naturel

Le problème

- Image : grille de **pixels** = nombres réels $\in [0, 1]$
- Texte : suite de **symboles discrets**
- Comment calculer la « dérivée » d'un mot ?
- Comment gérer la **polysémie** ?

Objectif du matin

Construire un **pont** entre le monde discret (mots) et le monde continu ($\mathbb{R}^d$), afin que les réseaux de neurones puissent traiter du texte.



Tokenisation (1/2) — Approches naïves

Niveau caractère

- \+ Petit vocabulaire ($\sim$256)
- \+ Aucun mot inconnu (OOV)
- – Perte de la sémantique locale
- – Séquences très longues

`chat` $\rightarrow$ `[c, h, a, t]`

Niveau mot (*word-level*)

- \+ Sens clair, séquences courtes
- – Vocabulaire immense ($V > 100\,000$)
- – Mots hors dictionnaire (OOV)
- – Flexions = tokens distincts

« mangeras » $\neq$ « mangeons »

Le dilemme

Caractère : trop fin — Mot : trop grossier $\Rightarrow$ Y a-t-il un juste milieu ?

Tokenisation (2/2) — Subword : Byte Pair Encoding

Principe du BPE

1. Initialiser : chaque caractère est un token
2. Compter les **paires** adjacentes les plus fréquentes
3. Fusionner la paire la plus fréquente → nouveau token
4. Répéter jusqu'à la taille de vocabulaire cible

Résultat

Mots **fréquents** conservés entiers.
Mots **rares** découpés en sous-mots significatifs.

Exemple

« inexplicablement »

↓

`[in | explicable | ment]`

Vocabulaires modernes

- GPT-2 : 50 257 tokens (BPE)
- BERT : 30 522 tokens (WordPiece)
- Couvrent **n'importe quel** texte

Vectorisation naïve : *One-Hot Encoding*

v_{chat}

0	roi
0	reine
1	**chat**
0	chien
0	lune
⋮	

Les 3 fléaux du One-Hot

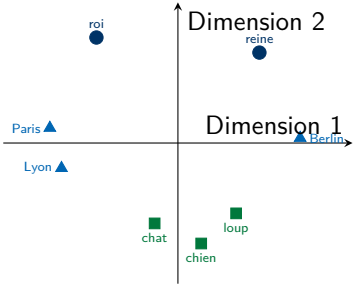
1. **Creusité** : $V = 50\,000$ valeurs, un seul 1
2. **Orthogonalité** : $v_{\text{chat}} \cdot v_{\text{chien}} = 0$ — aucune notion de proximité sémantique
3. **Dimensionnalité** : taille vecteur = taille vocabulaire

La solution : embedding dense

Représenter chaque token par un vecteur **dense** de petite dimension $d \ll V$:

$$d \in \{64,\ 128,\ 256,\ 768,\ \ldots\}$$

L'espace sémantique



Embedding dense $\in \mathbb{R}^d$

Chaque token est représenté par un vecteur réel :

$$\text{chat} \longmapsto \begin{bmatrix} 0.31 \\ -0.87 \\ \vdots \end{bmatrix} \in \mathbb{R}^{768}$$

Intuition géométrique

Les **dimensions** encodent des traits latents : genre, animité, géographie, abstraction...

Le sens devient une **position** dans l'espace : mots proches sémantiquement = proches géométriquement.

Apprendre l'espace : Word2Vec

L'hypothèse distributionnelle

« On connaît un mot par ses voisins. »

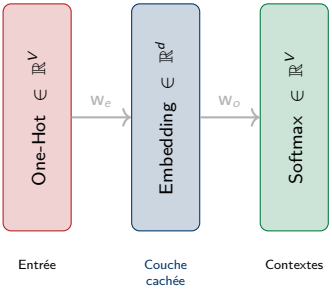
« le chat dort sur le tapis »

$\Rightarrow$ « chat » cooccure avec : dormir, tapis, ...

Méthode Skip-gram

1. Fenêtre glissante (taille k) sur le corpus
2. Réseau : prédire les mots **contexte** à partir du mot **cible**
3. Matrice de poids $\mathbf{W}_e \in \mathbb{R}^{V \times d}$ = dictionnaire d'embeddings

⋆ On **jette la prédiction** et ne conserve que $\mathbf{W}_e$



On conserve $\mathbf{W}_e$ après l'entraînement

Word2Vec : CBOW vs Skip-gram

CBOW (Continuous Bag of Words)

Entrée : mots contexte (moyennés)
Sortie : mot cible

[le, dort, sur] $\rightarrow$ `chat`

- Plus rapide à entraîner
- Meilleur sur grands corpus

Skip-gram

Entrée : mot cible
Sortie : mots contexte

`chat` $\rightarrow$ [le, dort, sur]

- Meilleur sur petits corpus
- Plus performant sur mots rares

Les deux apprennent la même chose

La tâche de prédiction est un **prétexte** : ce qui compte, c'est la matrice $\mathbf{W}_e \in \mathbb{R}^{V \times d}$ obtenue après entraînement. **Skip-gram** est le plus utilisé en recherche.

Entraîner Word2Vec efficacement : Negative Sampling

Le problème de la softmax

$$P(w \mid c) = \frac{e^{u_w \cdot v_c}}{\sum_{i=1}^{V} e^{u_i \cdot v_c}}$$

$V = 50\,000$: **calcul prohibitif** à chaque étape.

Solution

Pour chaque paire (w^+, c) positive :

- tirer k mots négatifs $w_1^-, \ldots, w_k^-$ au hasard
- maximiser $\sigma(u_{w^+} \cdot v_c)$
- minimiser $\sigma(u_{w_i^-} \cdot v_c)$

Mise à jour sur $k+1$ mots seulement. $k = 5$–20.

Exemple

Corpus : « le chat dort sur le tapis »

Paire positive : (*chat*, *dort*)

Paires négatives (aléatoires) :
(*chat*, *table*), (*chat*, *voiture*)...

Gain de vitesse $\approx \times V/k$.

Tirage pondéré

Probabilité $\propto f(w)^{3/4}$:
favorise légèrement les mots rares.

GloVe : co-occurrence globale

Idée centrale

Word2Vec : fenêtre glissante **locale**.

GloVe exploite la matrice de co-occurrence **globale** :

$$X_{ij} = \text{nb de fois que } j \text{ apparaît près de } i$$

Objectif : apprendre u_i, v_j tels que :

$$u_i \cdot v_j + b_i + b_j \approx \log X_{ij}$$

b_i, b_j : biais scalaires par mot (fréquence de base).
Les vecteurs encodent les **ratios** de co-occurrence.

Avantages

- Entraînement très rapide (une passe sur X)
- Capture les statistiques globales du corpus
- Analogies vectorielles aussi bonnes que Word2Vec

Vecteurs pré-entraînés

GloVe-6B (Wikipedia + Gigaword) :
400 000 mots, $d \in \{50, 100, 200, 300\}$.
Disponibles publiquement.

Algèbre de mots & similarité cosinus

Propriété : analogies vectorielles

Les relations sémantiques = translations dans $\mathbb{R}^d$:

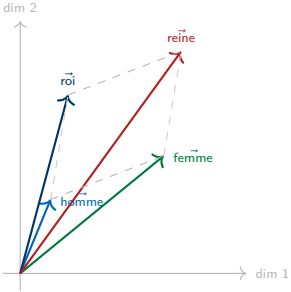
$$\vec{\text{roi}} - \vec{\text{homme}} + \vec{\text{femme}} \approx \vec{\text{reine}}$$

$$\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Allemagne}} \approx \vec{\text{Berlin}}$$

Mesure de proximité : similarité cosinus

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|} \in [-1, 1]$$

1 : synonymes 0 : sans rapport -1 : antonymes



Embeddings en PyTorch : nn.Embedding

```python
import torch, torch.nn as nn

# Matrice W_e : 10 000 mots, dim 128
emb = nn.Embedding(num_embeddings=10_000,
                   embedding_dim=128)

# Indices de tokens  [B=2, T=3]
idx = torch.tensor([[  3, 47, 892],
                    [ 12,  0,   1]])

x = emb(idx)  # [2, 3, 128] -- lookup differentiable

# --- Embeddings pre-entraines (GloVe / Word2Vec) ---
W = torch.FloatTensor(glove_matrix)  # [V, d]
emb = nn.Embedding.from_pretrained(W, freeze=False)
```

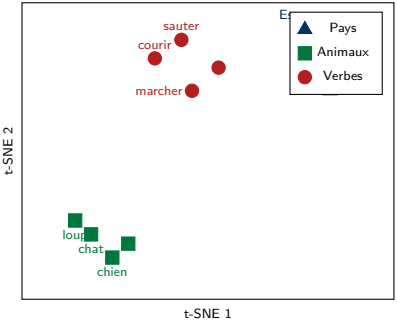
nn.Embedding en bref

- Matrice $\mathbf{W}_e \in \mathbb{R}^{V \times d}$ apprise par gradient
- emb(idx) : simple **lookup** de lignes
- Opération **différentiable**

freeze=False

Initialiser avec GloVe / Word2Vec, puis continuer l'entraînement : c'est du **transfer learning** au niveau mot.

Visualisation : réduction de dimension (t-SNE)



Le problème

Embeddings $\in \mathbb{R}^{768}$:
impossible à visualiser directement.

t-SNE / UMAP

- Projection en 2D
- **Préserve la structure locale** des voisinages
- Les clusters révèlent les **classes sémantiques**

Attention

t-SNE préserve la **proximité locale**, pas les distances globales.

Réduction de dimension : simplifier le modèle aval

Au-delà de la visualisation

Embeddings $\in \mathbb{R}^{768}$: espace de haute dimension.

Avec **PCA** (ou UMAP) : $\mathbb{R}^{768} \to \mathbb{R}^{50}$

$\Rightarrow$ un classifieur **simple** suffit souvent :

- Régression logistique
- SVM
- k-NN

Pipeline typique

`BERT / Word2Vec` $\to$ `PCA` $\to$ `LogisticRegression`

Avantages

- Entraînement **rapide**
- Moins de données nécessaires

Limite : surapprentissage

En haute dimension, le classifieur mémorise les données d'entraînement (**overfitting**).

La réduction de dimension **régularise** implicitement le modèle aval.

Limites des embeddings statiques

Le problème : la polysémie

- « Je dépose de l'argent à la **banque**. »
- « Je pêche au bord de la **banque**. »

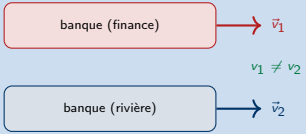
Dans Word2Vec / GloVe, « banque » possède **un seul vecteur**, mélangeant les deux sens.

Autres limites

- Ignorent l'ordre des mots
- Embeddings **figés** après entraînement
- Biais hérités des données d'entraînement

La solution : embeddings **contextuels**

Le vecteur d'un mot doit dépendre de son **contexte**.



J4-PM $\rightarrow$ mécanisme d'attention

L'attention permet de construire des embeddings **dynamiques** selon le contexte.

Encodage positionnel : vers les Transformers

Le problème

Les embeddings sont des **sacs de mots** :

« le chat mange la souris » $\equiv$ « la souris mange le chat »

$\Rightarrow$ Il faut encoder la **position** de chaque token.

Intuition

- Chaque position $\rightarrow$ vecteur unique $\in \mathbb{R}^d$
- Fréquences variées $\rightarrow$ positions proches = vecteurs similaires
- **Aucun paramètre** supplémentaire

Encodage sinusoïdal (Vaswani 2017)

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

On **additionne** à l'embedding : $x_t \leftarrow x_t + PE_t$

J4-PM $\rightarrow$ Transformers

L'encodage positionnel est la **première brique** de l'architecture Transformer.

Variante : PE appris (BERT, GPT).

Récapitulatif — Ce qu'on a vu

Tokenisation & représentations

- Niveaux : caractère / mot / sous-mot (BPE)
- One-Hot : creusité, orthogonalité, grande dim.
- Embedding dense $\in \mathbb{R}^d$: pont discret $\rightarrow$ continu
- Espace sémantique : proximité = sens partagé

En pratique

- `nn.Embedding` : lookup différentiable
- `from_pretrained` + `freeze`
- Encodage positionnel $\rightarrow$ Transformers (J4-PM)

Word2Vec & GloVe

- Hypothèse distributionnelle
- CBOW vs Skip-gram (tâche prétexte)
- Negative sampling : efficacité sur grand V
- GloVe : co-occurrence globale
- Analogies vectorielles, similarité cosinus
- t-SNE : visualisation des clusters
- Polysémie $\rightarrow$ embeddings contextuels (J4-PM)